

August 28, 2025

1 Temas Tratados en el Trabajo Práctico 3

- Estrategias de búsqueda local.
- Algoritmos Evolutivos.
- Problemas de Satisfacción de Restricciones.

2 Ejercicios Teóricos

1. ¿Qué mecanismo de detención presenta el algoritmo de Ascensión de Colinas? Describa el problema que puede presentar este mecanismo y cómo se llaman las áreas donde ocurren estos problemas.

El mecanismo de detención del algoritmo de Ascensión de Colinas ocurre cuando se llega a un estado donde ninguno de los vecinos mejora la solución actual. Es la obtención de un máximo, ya sea local o global. Un máximo local es una cima cuya altura es superior a la de sus estados vecinos, pero que es inferior a la cima más alta en todo el espacio de estados. Una vez que ha alcanzado un máximo local, el algoritmo se detiene, a pesar de que no se haya llegado a una solución, o al menos a una solución óptima. También puede detenerse en una meseta (zona donde varios estados tienen el mismo valor).

2. Describa las distintas heurísticas que se emplean en un problema de Satisfacción de Restricciones.

Los algoritmos de búsqueda se pueden definir aprovechándose de la estructura de los estados y utilizan las heurísticas de propósito general más que heurísticas específicas del problema para así permitir la solución de problemas grandes. Las heurísticas más comunes en problemas de Satisfacción de Restricciones (CSP) son:

Heurística de “mínimos valores restantes”: escoge la variable con menos valores legales. Heurística “valor menos restringido”: Selecciona primero la variable con menos valores posibles en su dominio, ya que es la más difícil de asignar. Heurística del grado: Si hay empate en MRV, elige la variable que tenga más restricciones con otras variables no asignadas.

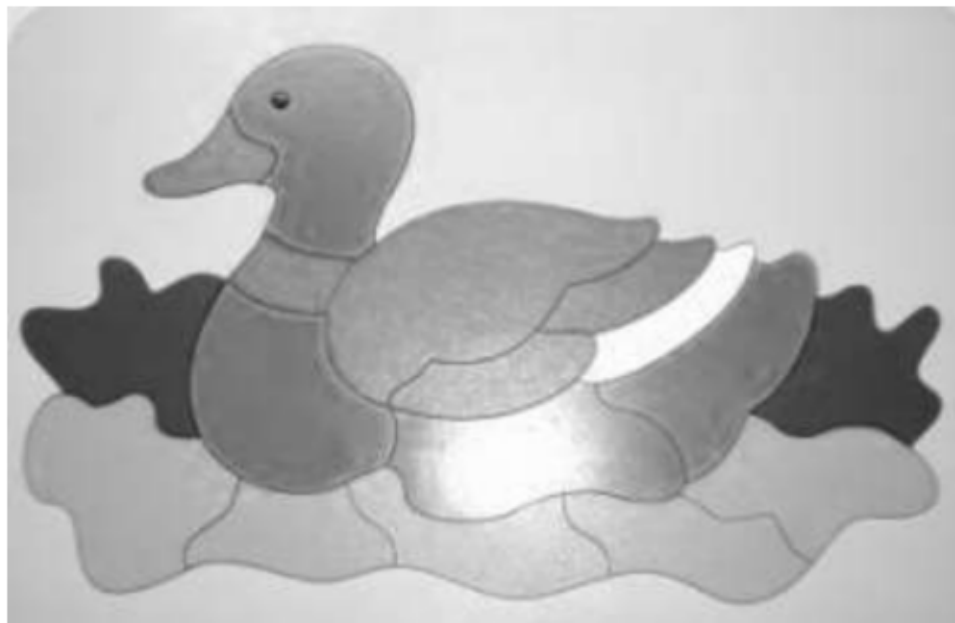
3. Se desea colorear el rompecabezas mostrado en la imagen con 7 colores distintos de manera que ninguna pieza tenga el mismo color que sus vecinas. Realice en una tabla el proceso de una búsqueda con Comprobación hacia Adelante empleando una heurística del Valor más Restringido.

```
[2]: import requests
      from PIL import Image
      from io import BytesIO
      import matplotlib.pyplot as plt

      # URL directa de Google Drive
      url = "https://drive.google.com/uc?
            ↪export=view&id=1j94jFVxVG9y_ZnrMW0scQGb2MZ0Cdb3R"

      # Descargar la imagen
      response = requests.get(url)
      img = Image.open(BytesIO(response.content))

      # Mostrar la imagen
      plt.imshow(img)
      plt.axis('off') # Ocultar ejes
      plt.show()
```



```
[ ]: from typing import Dict, Set, List, Tuple, Optional
      import random

      # =====
      # Definición del problema: Coloreo del rompecabezas del pato
      # =====
      # Colores disponibles (7, como indica el enunciado del TP)
```

```

COLORES_BASE: List[str] = ["Rojo", "Verde", "Azul", "Amarillo", "Naranja",
    ↪ "Violeta", "Rosa"]
# Grafo de adyacencias: cada pieza (nodo) y sus vecinos (con quienes comparte
    ↪ borde)
# IMPORTANTE: las piezas y vecinos fueron listados según tu descripción
GRAFO_ADYACENCIAS: Dict[str, Set[str]] = {
    "Pico": {"Cara", "Cielo"},
    "Cara": {"Ojos", "Cuello", "Pico", "Cielo", "Alas superior"},
    "Ojos": {"Cara"},
    "Cuello": {"Cara", "Pecho", "Alas superior", "Cielo"},
    "Pecho": {"Cuello", "Agua", "Tierra", "Alas superior", "Alas inferior"},
    "Alas superior": {"Cara", "Cuello", "Pecho", "Cielo", "Alas medias", "Alas
    ↪ inferior"},
    "Alas medias": {"Alas superior", "Alas inferior", "Cielo"},
    "Alas inferior": {"Alas superior", "Alas medias", "Cielo", "Tierra",
    ↪ "Agua"},
    "Agua": {"Pecho", "Alas inferior", "Tierra"},
    "Tierra": {"Agua", "Pecho", "Alas inferior", "Cielo"},
    "Cielo": {"Pico", "Cara", "Alas superior", "Alas medias", "Alas inferior",
    ↪ "Tierra"},
}

# =====
# Utilidades CSP (con aleatoriedad controlada)
# =====

def inicializar_dominios(nodos: List[str], colores_base: List[str]) ->
    ↪ Dict[str, List[str]]:
    """
    Inicializa dominios con orden aleatorio:
    - Se baraja una copia global de colores.
    - Se baraja el dominio de cada pieza para diversificar aún más.
    """
    colores = list(colores_base)
    random.shuffle(colores) # baraja el orden global de colores
    dominios = {}
    for n in nodos:
        d = list(colores)
        random.shuffle(d) # baraja también por pieza
        dominios[n] = d
    return dominios

def es_consistente(asig: Dict[str, str], var: str, valor: str, ady: Dict[str,
    ↪ Set[str]]) -> bool:
    return all(not (vec in asig and asig[vec] == valor) for vec in ady[var])

```

```

def mrv_variable(dominios: Dict[str, List[str]], asig: Dict[str, str]) -> str:
    """
    MRV con desempate aleatorio:
    - Encuentra el tamaño mínimo del dominio entre las no asignadas.
    - Si hay varias con ese mínimo, elige una al azar.
    - (Ligero sesgo adicional: preferir mayor grado, pero aún aleatorio entre
    ↪ candidatas)
    """
    no_asignadas = [v for v in dominios if v not in asig]
    # tamaño de dominio mínimo
    min_size = min(len(dominios[v]) for v in no_asignadas)
    candidatas = [v for v in no_asignadas if len(dominios[v]) == min_size]
    # Opcional: priorizar mayor grado dentro de candidatas y luego aleatorio
    max_grado = max(len(GRAFO_ADYACENCIAS[v]) for v in candidatas)
    candidatas = [v for v in candidatas if len(GRAFO_ADYACENCIAS[v]) ==
    ↪ max_grado]
    return random.choice(candidatas) # desempate aleatorio

def forward_checking(var: str, valor: str, dominios: Dict[str, List[str]], asig:
    ↪ Dict[str, str],
                                ady: Dict[str, Set[str]]) -> Tuple[bool, Dict[str,
    ↪ List[str]]]:
    """
    Comprobación hacia adelante (Forward Checking):
    - Elimina 'valor' del dominio de los vecinos no asignados.
    - Si algún vecino se queda sin valores, la asignación falla (retorna False).
    """
    nuevos = {v: list(vals) for v, vals in dominios.items()}
    for vecino in ady[var]:
        if vecino in asig:
            continue
        if valor in nuevos[vecino]:
            nuevos[vecino].remove(valor)
        if not nuevos[vecino]:
            return False, dominios
    return True, nuevos

# =====
# Algoritmo principal: Backtracking + MRV (aleatorio) + Forward Checking
# =====

def coloreo_csp(ady: Dict[str, Set[str]], colores_base: List[str]) -> Dict[str,
    ↪ str]:
    nodos = list(ady.keys())
    dominios = inicializar_dominios(nodos, colores_base)
    asig: Dict[str, str] = {}

```

```

def bt(doms: Dict[str, List[str]], a: Dict[str, str]) -> Optional[Dict[str, str]]:
    if len(a) == len(nodos):
        return a
    var = mrv_variable(doms, a)
    # Probar valores en el orden ya barajado del dominio de 'var'
    for valor in doms[var]:
        if not es_consistente(a, var, valor, ady):
            continue
        ok, nuevos = forward_checking(var, valor, doms, a, ady)
        if not ok:
            continue
        a[var] = valor
        res = bt(nuevos, a)
        if res is not None:
            return res
        del a[var] # backtrack
    return None

return bt(dominios, asig) or {}

# =====
# Ejecución
# =====

if __name__ == "__main__":
    # Si querés reproducibilidad en una corrida concreta, fija la semilla:
    # random.seed(123)
    solucion = coloreo_csp(GRAFO_ADYACENCIAS, COLORES_BASE)
    print("=== SOLUCIÓN (aleatoria por corrida) ===")
    for k, v in solucion.items():
        print(f"{k:15}: {v}")

```

```

=== SOLUCIÓN (aleatoria por corrida) ===
Cielo           : Rosa
Alas superior   : Azul
Cara            : Amarillo
Alas inferior   : Rojo
Alas medias     : Verde
Tierra          : Verde
Pecho           : Rosa
Cuello          : Rojo
Agua            : Violeta
Pico            : Rojo
Ojos            : Rosa

```

3 Ejercicios de Implementación

4. Encuentre el máximo de la función $f(x) = \frac{\sin(x)}{x+0.1}$ en $x \in [-10; -6]$ con un error menor a 0.1 utilizando el algoritmo *hill climbing*.

```
[83]: # Ascencion de Colinas con el error considerado como error posicional tal que,
# consideramos un paso fijo = 0.1, y un arranque aleatorio en cada ejecucion
# para ver como varia, entre [-10, -6].

import math, random

# Función: f(x) = sin(x) / (x + 0.1)
def f(x: float) -> float:
    return math.sin(x) / (x + 0.1)

def AscColina(f, rango=(-10.0, -6.0), paso=0.1):
    xmin, xmax = rango

    # El numero aleatorio inicial se toma dentro del intervalo.
    xaleat = random.uniform(xmin, xmax)

    # Redondeamos el valor de x para coincidir con el paso, o sea a un decimal.
    x = round(xaleat / paso) * paso

    # Aseguramos que x esté dentro del intervalo.
    x = max(min(x, xmax), xmin)

    # Valor de la funcion en ese punto.
    fx = f(x)

    # Determinamos el vecino siguiente y que no salga del intervalo.
    def vecino(xviejo, dx):
        xnuevo = max(min(xviejo + dx, xmax), xmin)
        return xnuevo, f(xnuevo)

    while True:
        xi, fi = vecino(x, -paso) # Vecino izquierdo.
        xd, fd = vecino(x, +paso) # Vecino derecho.

        # Si alguno de los vecinos mejora, nos movemos a ese vecino.
        if fi > fx and fi >= fd:
            x, fx = xi, fi
            continue
        elif fd > fx and fd >= fi:
            x, fx = xd, fd
            continue
        else:
```

```

        # Si no mejora es un máximo local.
        xi, fi = vecino(x, -paso)
        xd, fd = vecino(x, +paso)
        return {
            "Número inicial aleatorio": xaleat,
            "x max local": x,
            "f max local": fx,
            "Vecino izq": (xi, fi),
            "Vecino der": (xd, fd),
        }

res = AscColina(f, rango=(-10.0, -6.0), paso=0.1)

print(f'Número inicial aleatorio: {res["Número inicial aleatorio"]:.1f}')
print(f'x max local: {res["x max local"]:.1f}')
print(f'f max local: {res["f max local"]:.6f}')

xi, fi = res["Vecino izq"]
xd, fd = res["Vecino der"]
print(f'Vecino izq: (x = {xi:.1f}, f = {fi:.6f})')
print(f'Vecino der: (x = {xd:.1f}, f = {fd:.6f})')

```

```

Número inicial aleatorio: -7.2
x max local: -7.7
f max local: 0.130022
Vecino izq: (x = -7.8, f = 0.129681)
Vecino der: (x = -7.6, f = 0.129056)

```

5. Diseñe e implemente un algoritmo de Recocido Simulado para que juegue contra usted al Tarte-ti. Varíe los valores de temperatura inicial entre partidas, ¿qué diferencia observa cuando la temperatura es más alta con respecto a cuando la temperatura es más baja?

```

[3]: import math, random

#Definición de valores, siendo X el jugador 1, 0 el jugador 2 y E una casilla
↪vacía
X, 0, E = "X", "0", " "

#Definición de ternas ganadoras según las posiciones del tablero
WIN_LINES = [
    (0,1,2),(3,4,5),(6,7,8),
    (0,3,6),(1,4,7),(2,5,8),
    (0,4,8),(2,4,6)
]

# ----- Funciones de juego -----

```

```

#Mostrar el tablero en pantalla de forma que se vea como un tablero de Ta-te-ti
↳ 3x3, en lugar de una lista de 9 posiciones.
def print_board(board):
    """Imprime la grilla en formato 3x3"""
    print("\n")
# empieza en i = 0, llega hasta 9 (sin incluir el 9), avanza de a 3
    for i in range(0, 9, 3):
        print(" ", board[i], "|", board[i+1], "|", board[i+2])
        if i < 6:
            print(" ---+---+---")
    print("\n")

#Devuelve el ganador (X u O) o None si no hay ganador
def winner(board):
    for a,b,c in WIN_LINES:
        if board[a] != E and board[a] == board[b] == board[c]:
            return board[a]
    return None

#Devuelve una lista con las posiciones legales (vacías) del tablero
def legal_moves(board):
    return [i for i,v in enumerate(board) if v == E]

#Crea una copia del tablero, coloca la ficha (X u O) en la posición move y
↳ devuelve el tablero nuevo (sin modificar el original).
def apply_move(board, move, player):
    b = board.copy()
    b[move] = player
    return b

# ----- Función de coste -----

# Evalúa qué tan "bueno o malo" es un tablero desde la perspectiva de me (la
↳ IA).
#Si la IA ya ganó   coste = -1000 (muy bueno).
#Si el rival ganó   coste = +1000 (muy malo).
#Si nadie ganó:
#Si la IA está a 1 jugada de ganar   resta 10 (le gusta).
#Si el rival está a 1 jugada de ganar   suma 12 (le preocupa más).

def cost(board, me):
    opp = 0 if me == X else X
    w = winner(board)
    if w == me:
        return -1000
    if w == opp:
        return 1000

```



```

# Evaluar amenazas y oportunidades
score = 0
for (a,b,c) in WIN_LINES:
    line = [board[a], board[b], board[c]]
    if line.count(me) == 2 and line.count(E) == 1:
        score -= 10
    if line.count(opp) == 2 and line.count(E) == 1:
        score += 12
return score

# ----- Recocido Simulado -----

#T0 = temperatura inicial (qué tanto explora al principio).
#Tf = temperatura final (cuando termina).
#alpha = factor de enfriamiento (qué tan rápido baja la temperatura).

L = 30
def simulated_annealing(board, me, T0=5.0, Tf=0.1, alpha=0.95, L=30):
    moves = legal_moves(board)
    if not moves:
        return None

    # Solución inicial
    S_act = random.choice(moves)
    Cost_act = cost(apply_move(board, S_act, me), me)
    best_move, best_cost = S_act, Cost_act

    T = T0
    while T > Tf:
        for _ in range(L):
            S_cand = random.choice(moves)
            if S_cand == S_act:
                continue
            Cost_cand = cost(apply_move(board, S_cand, me), me)
            delta = Cost_cand - Cost_act

            if delta < 0 or random.random() < math.exp(-delta/T):
                S_act, Cost_act = S_cand, Cost_cand
                if Cost_act < best_cost:
                    best_move, best_cost = S_act, Cost_act
            T = alpha*T
        return best_move

# ----- Selección de dificultad -----
def elegir_dificultad():
    print("Selecciona dificultad:")

```

```

print("1. Fácil")
print("2. Medio")
print("3. Difícil")

while True:
    op = input("Elige (1-3): ")
    if op == "1":
        return 10.0    # Alta temperatura = IA más azarosa
    elif op == "2":
        return 5.0     # Temperatura media
    elif op == "3":
        return 1.0     # Baja temperatura = IA casi óptima
    else:
        print("Opción inválida, intenta de nuevo.")

# ----- Juego humano vs IA -----

#Crea el tablero vacío. Asigna fichas (O para humano, X para IA). Empieza la IA.
def play_game():
    board = [E]*9
    T0 = elegir_dificultad()

    # Asignar fichas
    human, ai = ("O", "X")

    # Sortear quién empieza
    turn = random.choice([human, ai])
    print(f"\n¡Bienvenido al Ta-te-ti con Recocido Simulado! (Dificultad_
↪seleccionada, T0={T0})")
    print(f"Empieza: {'Humano' if turn == human else 'IA'}")
    print("Casillas numeradas de la siguiente forma:")
    print(" 0 | 1 | 2 \n ---+---+---\n 3 | 4 | 5 \n ---+---+---\n 6 | 7 | 8 \n")

    while True:
        print_board(board)

        if winner(board) or not legal_moves(board):
            break

        if turn == human:
            try:
                mv = int(input("Tu turno (elige casilla 0-8): "))
            except:
                print("Entrada inválida.")
                continue
            if mv not in legal_moves(board):
                print("Movimiento no válido, intenta otra vez.")

```

```

        continue
    board[mv] = human
else:
    print("Turno de la IA...")
    mv = simulated_annealing(board, ai, T0=T0)
    board[mv] = ai
    print(f"IA juega en casilla {mv}")

turn = human if turn == ai else ai

print_board(board)
w = winner(board)
if w is None:
    print("¡Empate!")
elif w == human:
    print("¡Ganaste! ")
else:
    print("Gana la IA ")

# ----- Ejecutar -----
if __name__ == "__main__":
    play_game() # prueba cambiando T0=0.5, 3.0, 10.0

#Pasos dentro:
#Obtiene los movimientos legales.
#Elige una jugada inicial al azar.
#Calcula su coste.
#Guarda esa jugada como "mejor conocida" (best_move).
#Mientras T > Tf:
#Repite L veces:
#Elige una jugada candidata.
#Calcula su coste.
#Calcula la diferencia delta = Cost_cand - Cost_act.
#Acepta la candidata si:
#Es mejor (delta < 0).
#O es peor, pero con probabilidad exp(-delta/T).
#Si la jugada aceptada es la mejor hasta ahora, la guarda.
#Enfría: T = alpha * T.
#Devuelve la mejor jugada encontrada.

```

Selecciona dificultad:

1. Fácil
2. Medio
3. Difícil

¡Bienvenido al Ta-te-ti con Recocido Simulado! (Dificultad seleccionada, T0=1.0)
 Empieza: IA

Casillas numeradas de la siguiente forma:

```
0 | 1 | 2
---+---+---
3 | 4 | 5
---+---+---
6 | 7 | 8
```

```
  |  |
---+---+---
  |  |
---+---+---
  |  |
```

Turno de la IA...

IA juega en casilla 5

```
  |  |
---+---+---
  |  | X
---+---+---
  |  |
```

```
  |  | 0
---+---+---
  |  | X
---+---+---
  |  |
```

Turno de la IA...

IA juega en casilla 4

```
  |  | 0
---+---+---
  | X | X
---+---+---
  |  |
```

				0
----	+	----	+	----
0		X		X
----	+	----	+	----

Turno de la IA...

IA juega en casilla 8

				0
----	+	----	+	----
0		X		X
----	+	----	+	----
				X

0				0
----	+	----	+	----
0		X		X
----	+	----	+	----
				X

Turno de la IA...

IA juega en casilla 1

0		X		0
----	+	----	+	----
0		X		X
----	+	----	+	----
				X

0		X		0
----	+	----	+	----
0		X		X
----	+	----	+	----
0				X

0		X		0
-----+-----				
0		X		X
-----+-----				
0				X

¡Ganaste!

6. Diseñe e implemente un algoritmo genético para cargar una grúa con $n = 10$ *cajas* que puede soportar un peso máximo $C = 1000$ *kg*. Cada caja j tiene asociado un precio p_j y un peso w_j como se indica en la tabla de abajo, de manera que el algoritmo debe ser capaz de maximizar el precio sin superar el límite de carga.

Elemento (j)

1

2

3

4

5

6

7

8

9

10

Precio (p_j)

100

50

115

25

200

30

40

100

100

100

Peso (w_j)

300

200

450

145

664

90

150

355

401

395

6.1 En primer lugar, es necesario representar qué cajas estarán cargadas en la grúa y cuál

6.2 A continuación, genere una Población que contenga un número N de individuos (se recom

6.3 Cree ahora una función que permita evaluar la Idoneidad de cada individuo y seleccione

6.4 Por último, Cruce las parejas elegidas, aplique un mecanismo de Mutación y verifique q

6.5 Realice este proceso iterativamente hasta que se cumpla el mecanismo de detención de s

```
[15]: from dataclasses import dataclass
from typing import List, Tuple, Dict, Optional
import random

# PARÁMETROS GLOBALES
CAPACIDAD: int = 1000      # Capacidad fija
N: int = 20                # Cantidad de cajas

RANGO_PESO = (1, 1000)    # Pesos ~ U[1, 1000]
RANGO_PRECIO = (10, 100)  # Precios ~ U[10, 100]

# Parámetros del GA
SEED: Optional[int] = 42   # Semilla para reproducibilidad (None para variar en
    ↪ cada corrida)
TAM_POBLACION = 60
GENERACIONES = 200
PROB_UNO_INICIAL = 0.4
PC = 0.9                  # Probabilidad de cruce
PM = 1.0 / N              # Probabilidad de mutación por gen (1/N)
ELITISMO = 1              # N° de mejores que se copian directos a la sig.
    ↪ generación (>=1 recomendado)
```

```

MODULO_CONTROL = "penalizacion" # "penalizacion" ó "reparacion"
LAMBDA_PENAL = 1.0 # Coeficiente de castigo por exceso de peso (modo_
↳ "penalizacion")

# GENERACIÓN DE DATOS (precios y pesos)
def generar_listas(n: int, seed: Optional[int]) -> Tuple[List[int], List[int]]:
    """
    Genera listas paralelas de longitud n:
    - PRECIOS[j] en [10, 100]
    - PESOS[j] en [1, 1000]
    Sin correlación: instancia simple para el TP.
    """
    if seed is not None:
        random.seed(seed)
    precios = [random.randint(RANGO_PRECIO[0], RANGO_PRECIO[1]) for _ in
↳ range(n)]
    pesos = [random.randint(RANGO_PESO[0], RANGO_PESO[1]) for _ in
↳ range(n)]
    return precios, pesos

# Se generan una sola vez al cargar/ejecutar la celda
PRECIOS, PESOS = generar_listas(N, SEED)

# 6.1 - REPRESENTACIÓN: Individuo (cromosoma binario) + decodificación
@dataclass
class Individuo:
    """
    Cromosoma binario de longitud N:
    - genes[j] = 1 => tomar la caja j
    - genes[j] = 0 => no tomar la caja j
    """
    genes: List[int] # vector 0/1

    def precio_y_peso(self) -> Tuple[int, int]:
        """
        Decodifica el cromosoma: suma de precios y pesos de las cajas_
↳ seleccionadas.
        """
        precio_total = 0
        peso_total = 0
        for bit, p, w in zip(self.genes, PRECIOS, PESOS):
            if bit == 1:
                precio_total += p
                peso_total += w
        return precio_total, peso_total

    def es_factible(self) -> bool:

```



```

        """
        Verifica la restricción de capacidad fija (1000).
        """
        _, peso = self.precio_y_peso()
        return peso <= CAPACIDAD

def copia(self) -> "Individuo":
    """
    Devuelve una copia (para evitar aliasing en operadores).
    """
    return Individuo(self.genes[:])

def __str__(self) -> str:
    precio, peso = self.precio_y_peso()
    seleccion = [i+1 for i, b in enumerate(self.genes) if b == 1]
    return (f"Individuo(genes={self.genes}) | Cajas={seleccion} | "
            f"Precio={precio} | Peso={peso} | Factible={self.
↪es_factible()}")

# Constructores útiles
def individuo_vacio() -> Individuo:
    """Cromosoma todo 0: siempre factible."""
    return Individuo([0]*N)

def individuo_lleno() -> Individuo:
    """Cromosoma todo 1: normalmente infactible, útil para testear."""
    return Individuo([1]*N)

def individuo_aleatorio(prob_uno: float = PROB_UNO_INICIAL) -> Individuo:
    """Cromosoma aleatorio (Bernoulli por gen)."""
    genes = [1 if random.random() < prob_uno else 0 for _ in range(N)]
    return Individuo(genes)

# 6.2 - EVALUACIÓN: penalización o reparación
def evaluar_penalizado(ind: Individuo, lam: float = LAMBDA_PENAL) -> Dict:
    """
    Fitness con PENALIZACIÓN lineal por exceso de peso.
    - Si factible: fitness = precio_total
    - Si excede: fitness = precio_total - lam*(peso - CAPACIDAD)
    Devuelve dict con métricas útiles.
    """
    precio, peso = ind.precio_y_peso()
    exceso = max(0, peso - CAPACIDAD)
    fitness = precio - lam * exceso
    return {
        "individuo": ind,
        "precio": precio,

```

```

        "peso": peso,
        "exceso": exceso,
        "fitness": fitness,
        "factible": exceso == 0
    }

def reparar_aleatorio(ind: Individuo) -> Individuo:
    """
    REPARACIÓN mínima y simple:
    Mientras el peso supere la capacidad, apaga (pone a 0) aleatoriamente
    alguno de los genes que están en 1. Sin heurísticas de ratio.
    """
    genes = ind.genes[:]
    while True:
        precio, peso = Individuo(genes).precio_y_peso()
        if peso <= CAPACIDAD:
            return Individuo(genes)
        activos = [i for i, b in enumerate(genes) if b == 1]
        if not activos:
            return Individuo([0]*N) # borde: si quedó vacío
        i = random.choice(activos)
        genes[i] = 0

def evaluar_por_reparacion(ind: Individuo) -> Dict:
    """
    Fitness con REPARACIÓN:
    - Si infactible, primero reparar_aleatorio(ind), luego fitness = precio del
    ↪reparado.
    - Siempre devuelve un individuo factible.
    """
    ind_ok = reparar_aleatorio(ind) if not ind.es_factible() else ind
    precio, peso = ind_ok.precio_y_peso()
    return {
        "individuo": ind_ok,
        "precio": precio,
        "peso": peso,
        "exceso": 0,
        "fitness": precio,
        "factible": True
    }

def evaluar(ind: Individuo, modo: str = MODO_CONTROL) -> Dict:
    """
    Despacha la evaluación según el modo de control de restricciones elegido.
    """
    if modo == "penalizacion":
        return evaluar_penalizado(ind)

```

```

elif modo == "reparacion":
    return evaluar_por_reparacion(ind)
else:
    raise ValueError("modo debe ser 'penalizacion' o 'reparacion'")

def generar_poblacion(tam: int) -> List[Dict]:
    """
    Población inicial + evaluación según el modo seleccionado.
    Retorna lista de dicts con (individuo, precio, peso, exceso, fitness,
    ↪ factible).
    """
    resultados = []
    for _ in range(tam):
        ind = individuo_aleatorio(PROB_UNO_INICIAL)
        resultados.append(evaluar(ind, MODO_CONTROL))
    return resultados

# 6.3 - SELECCIÓN por ruleta (fitness-proportionate)
def seleccionar_ruleta(pobl_ev: List[Dict]) -> Individuo:
    """
    Selección proporcional al fitness (ruleta). Para evitar problemas con
    ↪ fitness
    negativos (caso penalización), se aplica un corrimiento.
    Devuelve una copia del Individuo seleccionado.
    """
    fitnesses = [r["fitness"] for r in pobl_ev]
    # Corrimiento para garantizar no-negatividad
    min_fit = min(fitnesses)
    shift = -min_fit + 1e-9 if min_fit <= 0 else 0.0
    pesos = [f + shift for f in fitnesses]
    total = sum(pesos)
    # Si por algún motivo todos son ~0 (p. ej., penalización muy fuerte),
    ↪ fallback uniforme
    if total <= 1e-12:
        elegido = random.choice(pobl_ev)["individuo"].copia()
        return elegido
    # Ruleta
    r = random.random() * total
    acum = 0.0
    for rdoc in pobl_ev:
        acum += (rdoc["fitness"] + shift)
        if acum >= r:
            return rdoc["individuo"].copia()
    # Por redondeos, devolvemos el último
    return pobl_ev[-1]["individuo"].copia()

# 6.4 - CRUCE y MUTACIÓN

```

```

def cruce_un_punto(p1: Individuo, p2: Individuo, pc: float = PC) -> Tuple[Individuo, Individuo]:
    """
    Cruce de 1 punto (con probabilidad pc). Si no cruza, los hijos quedan iguales a los padres.
    """
    if random.random() >= pc or N < 2:
        return p1.copia(), p2.copia()
    corte = random.randint(1, N-1) # punto de corte entre [1, N-1]
    h1_genes = p1.genes[:corte] + p2.genes[corte:]
    h2_genes = p2.genes[:corte] + p1.genes[corte:]
    return Individuo(h1_genes), Individuo(h2_genes)

def mutacion_bitflip(ind: Individuo, pm: float = PM) -> Individuo:
    """
    Mutación por inversión de bit (flip) con probabilidad pm por gen.
    """
    genes = ind.genes[:]
    for i in range(N):
        if random.random() < pm:
            genes[i] = 1 - genes[i]
    return Individuo(genes)

# 6.5 - BUCLE EVOLUTIVO (elitismo + selección + cruce + mutación)
def mejor_de(pobl_ev: List[Dict]) -> Dict:
    """
    Devuelve el dict con mejor fitness en 'pobl_ev'.
    """
    return max(pobl_ev, key=lambda r: r["fitness"])

def evolucionar(
    generaciones: int = GENERACIONES,
    tam_pob: int = TAM_POBLACION,
    pc: float = PC,
    pm: float = PM,
    elitismo: int = ELITISMO,
    modo: str = MODO_CONTROL
) -> Dict:
    """
    Bucle principal del GA:
    - Genera población inicial
    - Repite: seleccionar, cruzar, mutar, evaluar, elitismo
    - Lleva registro del mejor global y del historial de fitness
    Retorna un diccionario con resultados (mejor, historial, y parámetros).
    """
    assert tam_pob % 2 == 0, "Para simplificar, usá tamaño de población par."
    pobl_ev = generar_poblacion(tam_pob)

```

```

hist_best = []
best_global = mejor_de(pobl_ev)

for g in range(generaciones):
    # 1) ELITISMO: copiar los 'elitismo' mejores directamente
    pobl_ev_ordenada = sorted(pobl_ev, key=lambda r: r["fitness"],
    ↪reverse=True)
    next_gen_inds: List[Individuo] = [rdoc["individuo"].copia() for rdoc in
    ↪pobl_ev_ordenada[:elitismo]]

    # 2) Rellenar la nueva generación con hijos de selección + cruce +
    ↪mutación
    while len(next_gen_inds) < tam_pob:
        p1 = seleccionar_ruleta(pobl_ev)
        p2 = seleccionar_ruleta(pobl_ev)
        h1, h2 = cruce_un_punto(p1, p2, pc=pc)
        h1 = mutacion_bitflip(h1, pm=pm)
        h2 = mutacion_bitflip(h2, pm=pm)
        next_gen_inds.append(h1)
        next_gen_inds.append(h2)

    # 3) Evaluar la nueva generación
    pobl_ev = [evaluar(ind, modo=modo) for ind in next_gen_inds]

    # 4) Actualizar bests e historial
    best_gen = mejor_de(pobl_ev)
    if best_gen["fitness"] > best_global["fitness"]:
        best_global = best_gen
    hist_best.append(best_gen["fitness"])

return {
    "mejor_global": best_global,
    "historial_mejor": hist_best,
    "params": {
        "N": N, "CAPACIDAD": CAPACIDAD,
        "TAM_POBLACION": tam_pob, "GENERACIONES": generaciones,
        "PC": pc, "PM": pm, "ELITISMO": elitismo, "MODO": modo,
        "SEED": SEED
    }
}

if __name__ == "__main__":
    # Muestra datos de la instancia
    print("=== INSTANCIA ===")
    print(f"N = {N}, CAPACIDAD = {CAPACIDAD}, SEED = {SEED}")
    print(f"PRECIOS = {PRECIOS}")

```

```

print(f"PESOS    = {PESOS}")

# Corre el GA
resultados = evolucionar(
    generaciones=GENERACIONES,
    tam_pob=TAM_POBLACION,
    pc=PC, pm=PM,
    elitismo=ELITISMO,
    modo=MODO_CONTROL
)

best = resultados["mejor_global"]["individuo"]
best_precio = resultados["mejor_global"]["precio"]
best_peso = resultados["mejor_global"]["peso"]
best_fit = resultados["mejor_global"]["fitness"]
hist = resultados["historial_mejor"]

print("\n=== RESULTADO FINAL ===")
print(best)
print(f"Mejor Precio = {best_precio}")
print(f"Peso          = {best_peso} (<= {CAPACIDAD} ? {'SI' if_
↪best_peso<=CAPACIDAD else 'NO'})")
print(f"Fitness      = {best_fit}")
print("\n=== PARÁMETROS ===")
for k, v in resultados["params"].items():
    print(f"{k}: {v}")

# Resumen básico de la evolución (sin gráficos)
print("\n=== EVOLUCIÓN (resumen) ===")
print(f"Fitness mejor por generación (primeras 10): {hist[:10]}")
if len(hist) > 10:
    print(f"... (últimas 10): {hist[-10:]}")

```

=== INSTANCIA ===

N = 20, CAPACIDAD = 1000, SEED = 42

PRECIOS = [91, 24, 13, 45, 41, 38, 27, 23, 96, 79, 21, 85, 64, 14, 13, 21, 37, 39, 74, 87]

PESOS = [28, 575, 204, 734, 666, 719, 559, 430, 226, 460, 604, 285, 829, 891, 7, 778, 826, 164, 715, 433]

=== RESULTADO FINAL ===

Individuo(genes=[1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1]) |

Cajas=[1, 9, 12, 15, 20] | Precio=372 | Peso=979 | Factible=True

Mejor Precio = 372

Peso = 979 (<= 1000 ? SI)

Fitness = 372.0

=== PARÁMETROS ===

N: 20

CAPACIDAD: 1000

TAM_POBLACION: 60

GENERACIONES: 200

PC: 0.9

PM: 0.05

ELITISMO: 1

MOD0: penalizacion

SEED: 42

=== EVOLUCIÓN (resumen) ===

Fitness mejor por generación (primeras 10): [-382.0, 53.0, 53.0, 127.0, 149.0, 149.0, 212.0, 212.0, 212.0, 238.0]

... (últimas 10): [372.0, 372.0, 372.0, 372.0, 372.0, 372.0, 372.0, 372.0, 372.0, 372.0]

4 Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada